

PROPOSAL PROYEK AKHIR

A. Judul

Analisis Efektivitas Implementasi *Self-Healing* menggunakan Playwright MCP dalam Mengurangi Beban *Maintenance* pada *Automation Testing*.

B. Deskripsi

Proses *Maintenance* skrip *automation testing* seringkali menjadi tantangan selama proses pengembangan, khususnya pada metodologi *agile* yang seringkali melakukan perubahan pada perangkat lunak (Kirinuki et al., 2019). Penyebab permasalahan ini salah satunya adalah *locators*. Ketika *developer* memperbarui atribut sebuah *website*, skrip tes yang bersangkutan juga harus dimodifikasi, sehingga menyebabkan proses pemeliharaan secara berkelanjutan dan memakan banyak waktu (Samad et al., 2021). Beberapa faktor lain juga berperan menghambat proses pemeliharaan, seperti gangguan jaringan, *flaky test*, hingga keterampilan dari *tester* sendiri. Mengatasi masalah ini, adanya kemampuan untuk memperbaiki sendiri kode secara otomatis (*self-healing*) menjadi cara efektif untuk mengurangi waktu dan usaha dalam pemeliharaan skrip *automation* (Yunjin Kim & Rand Kouatly, 2024).

Self-healing skrip *automation* dilakukan dengan memanfaatkan Playwright MCP. Playwright dipilih karena mampu menjalankan *automation testing* secara *end-to-end* dengan baik. Setelah skrip *automation* berhasil dibuat, mekanisme *self-healing* dibangun dengan memanfaatkan *AI Agent* yang dijalankan melalui MCP menggunakan *prompt* berbahasa alami sebagai konteks. Proses ini akan dieksekusi ketika terjadi kegagalan selama eksekusi skrip *automation* dengan menjalankan, memperbaiki, dan mengubah skrip yang telah dikembangkan sesuai dengan konteks *prompt* yang diberikan.

Pengujian dilakukan dengan mensimulasikan skenario *error* seperti perubahan *locator*, *flaky test*, *timeout*, serta penambahan fitur baru. Skenario ini kemudian akan dijalankan secara berulang menggunakan variasi *model LLM* dan metode *prompting* yang berbeda sehingga dapat dianalisis pengaruhnya terhadap kemampuan *self-healing*. Evaluasi dilakukan dengan melihat kemampuan dari Playwright MCP pada metrik ukur seperti *recovery success rate*, *execution time*, serta melihat efektivitas *model* dalam mengatasi *flaky test*. Dengan demikian, pengujian ini diharapkan mampu melihat kemampuan dari Playwright MCP dalam melakukan *self-healing*.

C. Tools & Tech Stack

Proyek ini merupakan proyek *Quality Assurance* yang dikembangkan menggunakan:

- Playwright (Typescript): framework utama untuk *automation testing* dengan dukungan *Model Context Protocol* (MCP).
- *LLM Agent/Model*: digunakan untuk mendukung mekanisme *self-healing* berbasis *AI* yang terintegrasi dengan Playwright MCP.
- Copilot Chat (VS Code) digunakan untuk membantu menulis, memperbaiki, dan menjalankan skrip *self-healing* secara interaktif
- Allure Report/Playwright Test Report: evaluasi hasil eksekusi *automation testing*.

D. Mitra / Perusahaan

Lab RPL

E. Daftar Pustaka

- Samad, A., Nafis, M. T., Rahmani, M. S., & Sohail, S. S. (2021). *A Cognitive Approach in Software Automation Testing*. ICICC 2021. SSRN. <https://ssrn.com/abstract=3834262>
- Kirinuki, H., Tanno, H., & Natsukawa, K. (2019). *COLOR: Correct Locator Recommender for Broken Test Scripts using Various Clues in Web Application*. IEEE Transactions on Software Engineering. <https://doi.org/10.1109/SANER.2019.8667976>
- Kim, Y., & Kouatly, R. (2024). *Self-healing automation testing with Selenium and ChatGPT API integration to improve the efficiency of the maintenance*. 2nd International Conference on Foundation and Large Language Models (FLLM2024). IEEE. <https://doi.org/10.1109/FLLM63129.2024.10852490>
- Ramdhani, M. D., Setyanto, A., & Ariatmanto, D. (2024). *Optimalisasi Proses Pembaharuan Locator pada Kode Otomatis Selenium dengan Menggunakan Page Object Model*. JIKA (Journal of Informatics), 8(1), 33–42. Universitas Muhammadiyah Tangerang. <https://doi.org/10.31000/jika.v8i1.9710>
- Collins, E. F., & Lucena Jr, V. F. de. (2012). *Software Test Automation Practices in Agile Development Environment: An Industry Experience Report*. IEEE International Conference on Agile Software Development (AST 2012), Zurich, Switzerland. <https://doi-org.ezproxy.ugm.ac.id/10.1109/IWAST.2012.6228991>